TÀI LIỆU ĐẶC TẢ YÊU CẦU PHẦN MỀM
(Software Requirement Specification – SRS)

Dự án: Phát triển nền tảng Low-code hỗ trợ xây dựng và vận hành chiến lược giao dịch tiền mã hóa trên sàn giao dịch tập (CEX)

Tên sản phẩm: QUANTFLOW

Phiên bản 1.0

Được soạn bởi Khanh

Cần Thơ, tháng 2, năm 2026

Mục lục

1 Giới thiệu 6
1.1 Mục đích 6
1.2 Phạm vi dự án 6
1.2.1 Tổng quan sản phẩm 6
1.2.2 Các chức năng trong phạm vi (In-scope) 6
1.2.3 Các chức năng ngoài phạm vi (Out-of-scope) 7
1.3 Định nghĩa và thuật ngữ 7
2 Yêu cầu chức năng 8
2.1 Phân hệ Truy cập & Cấu hình 8
2.2 Phân hệ Thiết kế chiến lược giao dịch 9
2.3 Phân hệ Backtest & Live Trade 12
2.3.1 Chức năng Backtest 12
2.3.1 Chức năng Live Trade 13
2.4 Phân hệ theo dõi 15
2.5 Phân hệ Lõi: Tích hợp & Quản lý dữ liệu 16
3 Yêu cầu phi chức năng 18
3.1 Yêu cầu hiệu năng 18
3.2 Bảo mật 19
3.3 Độ tin cậy 20
4 Biểu đồ use case 21
4.1 Biểu đồ use case tổng quan 21
4.2 Đặc tả use case 22
4.2.1 UC-01: Đăng nhập (Login) 22
4.2.2 UC-02: Cập nhật thông tin tài khoản (Update Account Info) 24
4.2.3 UC-03: Cấu hình API Sàn (Configure API Keys) 26
4.2.4 UC-04: Thiết kế chiến lược (Design Strategy) 28
4.2.5 UC-05: Quản lý mẫu chiến lược (Manage Strategy Templates) 31
4.2.6 UC-06: Thực thi Backtest (Run Backtest) 33
4.2.7 UC-08: Khởi tạo & Điều khiển Bot (Manage Bot Instance) 35
4.2.8 UC-09: Giám sát Bot đang chạy (Monitor Live Bot) 37

4.2.9 UC-10: Xem thị trường (View Market) 39
4.2.10 UC-11: Xem lịch sử giao dịch (View Trade History) 41

DANH SÁCH BẢNG

Bảng 1-1: Từ điển thuật ngữ 7
Bảng 2-1: Yêu cầu chức năng Phân hệ Truy cập & Cấu hình..... 8
Bảng 2-2: Yêu cầu chức năng Phân hệ Thiết kế chiến lược giao dịch.....11
Bảng 2-3: Yêu cầu chức năng Backtest 13
Bảng 2-4: Yêu cầu chức năng Live Trade..... 14
Bảng 2-5: Yêu cầu chức năng Phân hệ theo dõi 15
Bảng 2-6: Yêu cầu chức năng Phân hệ Lõi..... 17
Bảng 3-1: Yêu cầu chức năng Phân hệ Truy cập & Cấu hình..... 18
Bảng 3-2: Yêu cầu phi chức năng Bảo mật..... 19
Bảng 3-3: Yêu cầu phi chức năng Độ tin cậy 20
Bảng 4-1: Đặc tả UC-01: Đăng nhập (Login)..... 23
Bảng 4-2: Đặc tả UC-02: Cập nhật thông tin tài khoản (Update Account Info)..... 25
Bảng 4-3: Đặc tả UC-03: Cấu hình API Sàn (Configure API Keys) 27
Bảng 4-4: Đặc tả UC-04: Thiết kế chiến lược (Design Strategy) 30
Bảng 4-5: Đặc tả UC-05: Quản lý mẫu chiến lược (Manage Strategy Templates)..... 32
Bảng 4-6: Đặc tả UC-06: Thực thi Backtest (Run Backtest)..... 34
Bảng 4-7: Đặc tả UC-08: Khởi tạo & Điều khiển Bot (Manage Bot Instance) 36
Bảng 4-8: Đặc tả UC-09: Giám sát Bot đang chạy (Monitor Live Bot)..... 38
Bảng 4-9: Đặc tả UC-10: Xem thị trường (View Market)..... 40
Bảng 4-10: Đặc tả UC-11: Xem lịch sử giao dịch (View Trade History)..... 42

DANH SÁCH HÌNH

Hình 4-1: Biểu đồ use case tổng quan 21

1 Giới thiệu

1.1 Mục đích

Mục đích của tài liệu này là xác định đầy đủ và chi tiết các yêu cầu phần mềm cho đề tài "Phát triển nền tảng Low-code hỗ trợ xây dựng và vận hành chiến lược giao dịch tiền mã hóa trên sàn giao dịch tập trung (CEX)".

Tài liệu này được soạn thảo nhằm đạt được các mục tiêu cụ thể sau:

- Thống nhất phạm vi: Xác định rõ ràng các chức năng hệ thống sẽ thực hiện và các giới hạn của hệ thống.
- Định hướng phát triển: Làm cơ sở kỹ thuật để xây dựng kiến trúc hệ thống (System Architecture).
- Căn cứ kiểm thử: Cung cấp các tiêu chí cụ thể để xây dựng kế hoạch kiểm thử (Test Plan) và các trường hợp kiểm thử (Test Cases), đảm bảo sản phẩm hoạt động đúng như mong đợi.

1.2 Phạm vi dự án

1.2.1 Tổng quan sản phẩm

Sản phẩm là một ứng dụng web (Web Application) cung cấp môi trường lập trình trực quan (Low-code/No-code) cho phép người dùng cá nhân tự xây dựng, cấu hình và vận hành các bot giao dịch tiền mã hóa tự động trên các sàn giao dịch tập trung (CEX), cụ thể là Binance.

1.2.2 Các chức năng trong phạm vi (In-scope)

Hệ thống sẽ bao gồm các nhóm chức năng cốt lõi sau:

- Quản trị truy cập & Cấu hình (Access & Configuration):
- Hệ thống hỗ trợ mô hình đơn người dùng (Single User/Admin).
- Cho phép người dùng đăng nhập, đổi tên đăng nhập và thay đổi mật khẩu cá nhân.
- Quản lý an toàn các khóa API (API Key/Secret Key) kết nối với sàn giao dịch.

Xây dựng chiến lược (Strategy Builder):

- Cung cấp giao diện kéo thả (Drag-and-drop) sử dụng thư viện Google Blockly.
- Cung cấp bộ công cụ khối logic đầy đủ: Điều kiện (If/Else), Vòng lặp, Toán học, Hàm chỉ báo, và các khối chức năng giao dịch (Mua/Bán/Hủy lệnh).
- Khả năng chuyển đổi từ sơ đồ khối sang mã thực thi (Code Generation).

Vận hành & Giám sát (Execution & Monitoring):

- Cơ chế Unit Cost: Kiểm soát tài nguyên thực thi của từng khối lệnh để ngăn chặn vòng lặp vô tận hoặc treo hệ thống.
- Khởi chạy và dừng bot theo yêu cầu.
- Hiển thị nhật ký hoạt động (Logs) và trạng thái lệnh theo thời gian thực (Real-time) qua WebSocket.

Kết nối thị trường:

- Lấy dữ liệu giá thị trường (Market Data) từ sàn CEX.
- Gửi lệnh giao dịch (Order Placement) thông qua API của sàn.

1.2.3 Các chức năng ngoài phạm vi (Out-of-scope)

Để đảm bảo tính khả thi và tập trung vào chất lượng của các tính năng cốt lõi trong thời gian giới hạn, các hạng mục sau sẽ KHÔNG được phát triển trong phiên bản này:

- Không hỗ trợ Sàn phi tập trung (DEX): Hệ thống không kết nối với Blockchain trực tiếp hay các sàn như Uniswap, PancakeSwap (do phức tạp về Smart Contract).
- Không phát triển Ứng dụng di động (Mobile App): Sản phẩm chỉ hoạt động trên nền tảng Web (Browser).
- Không có tính năng Mạng xã hội: Không hỗ trợ chia sẻ chiến lược, copy-trade, hay mua bán bot giữa các người dùng khác nhau.
- Không tích hợp cổng thanh toán: Hệ thống là công cụ cá nhân, không tích hợp nạp/rút tiền pháp định (Fiat) hay thu phí người dùng.
- Giới hạn về AI: Mặc dù hệ thống hỗ trợ logic phức tạp, việc tích hợp các mô hình Deep Learning (như Reinforcement Learning) để tự trade là một hướng phát triển tương lai, không nằm trong cam kết hoàn thành của phiên bản hiện tại (trừ khi được phát triển dưới dạng module thử nghiệm riêng biệt).

1.3 Định nghĩa và thuật ngữ

CEX	Sàn giao dịch tiền ảo tập chung (Centralized Exchange)
Backtest	Kiểm thử chiến lược giao dịch bằng các sử dụng dữ liệu giao dịch trong quá khứ
Live Trade	Vận hành chiến lược giao dịch trên thị trường thực tế
Futures	Giao dịch hợp đồng tương lai
Session	Phiên chạy của chiến lược giao dịch được tính bằng một lần gọi khối sự kiện sẽ là một sesssion
Unit	Giá trị của mỗi khối logic trong khối lệnh trong vận hành chiến lược giao dịch
Binance	Là sàn giao dịch tiền mã hóa tập chung

Bảng 1-1: Từ điển thuật ngữ

2 Yêu cầu chức năng

2.1 Phân hệ Truy cập & Cấu hình

Mã yêu cầu	Tên chức năng	Mô tả chi tiết	Ghi chú kỹ thuật
FR-ACCESS-01	Đăng nhập hệ thống	Người dùng đăng nhập bằng Tên đăng nhập và Mật khẩu.	Nếu sai quá 5 lần, khóa đăng nhập trong 15 phút (Brute-force protection).
FR-ACCESS-02	Cập nhật thông tin tài khoản	Cho phép người dùng thay đổi: 1. Tên đăng nhập mới. 2. Mật khẩu mới. Để thực hiện thay đổi, người dùng bắt buộc phải nhập đúng Mật khẩu hiện tại để xác thực.	Sau khi đổi Username hoặc Password thành công, hệ thống nên tự động đăng xuất (Logout) và yêu cầu đăng nhập lại với thông tin mới để đảm bảo an toàn.
FR-ACCESS-03	Cấu hình API Key Sàn	Người dùng nhập và lưu API Key / Secret Key của sàn giao dịch.	Mã hóa AES-256 cho Secret Key.
FR-ACCESS-04	Đăng xuất	Kết thúc phiên làm việc.	Xóa Cookie kết thúc Session.

Bảng 2-1: Yêu cầu chức năng Phân hệ Truy cập & Cấu hình

2.2 Phân hệ Thiết kế chiến lược giao dịch

Mã Yêu cầu	Tên chức năng	Mô tả chi tiết	**Ghi chú kỹ thuật**
FR-DESIGN-01	Không gian làm việc (Workspace)	Vùng thao tác vô hạn để thiết kế logic. Hỗ trợ Zoom in/out, Undo/Redo, Xóa khối, Căn chỉnh khối lệnh.	Thư viện: Google Blockly.
FR-DESIGN-02	Thanh công cụ (Toolbox)	Danh sách các khối lệnh phân theo nhóm màu: *Sự kiện, Logic, Biến, Toán học, Chỉ báo, Giao dịch*.	
FR-DESIGN-03	Khối Sự kiện (Event Trigger)	Khối khởi đầu (Start Block). Định nghĩa thời điểm kích hoạt một phiên xử lý (Session). - Input: Khung thời gian (1m, 5m, 15m, 1h...). - Loại sự kiện: *Khi đóng nến* (On Candle Close) hoặc *Khi mở nến* (On Candle Open).	Backend đăng ký lắng nghe WebSocket. Khi sự kiện xảy ra, logic bên trong khối này được thực thi 1 lần.
FR-DESIGN-04	Quản lý Biến (Variable Scopes)	Hỗ trợ khởi tạo và sử dụng 2 loại biến với vòng đời khác nhau: 1. Biến Phiên (Session Variable): Chỉ tồn tại trong 1 lần kích hoạt sự kiện, tự hủy khi kết thúc phiên. 2. Biến Vòng đời (Lifecycle Variable): Lưu giá trị vĩnh viễn trong bộ nhớ suốt thời gian Bot chạy (giữ giá trị từ phiên này sang phiên khác).	**Frontend:** Phân biệt bằng màu sắc hoặc prefix (VD: sess_ vs life_). **Backend:** Biến Lifecycle lưu trong Context của Bot Instance.
FR-DESIGN-05	Khối Điều khiển & Logic	Các khối điều hướng luồng chương trình: - Điều kiện: If, If-Else.	

		- Logic: And, Or, Not, So sánh (=, >, <). - Vòng lặp: Repeat, While.	
FR-DESIGN-06	Khối Toán học cơ bản	Các phép tính số học phục vụ tính toán giá/khối lượng: - Cơ bản: +, -, *, /. - Nâng cao: Làm tròn (Round), Random.	- Chỉ hỗ trợ số thực/nguyên. - Sử dụng BigInt trong code
FR-DESIGN-07	Khối Chỉ báo Kỹ thuật (Context-aware)	Tính toán chỉ báo cho cặp tiền hiện tại đang chạy: - RSI: Input (Timeframe, Period). - EMA: Input (Timeframe, Period).	Logic tự động lấy dữ liệu nến của *Current_Symbol* để tính toán.
FR-DESIGN-08	Khối Dữ liệu Thị trường & Tài khoản	Truy xuất dữ liệu thời gian thực của cặp tiền đang chạy: - Get Market Price: Giá hiện tại/Giá đóng nến. - Get Position Info: Vị thế hiện tại (Size, PnL, Entry Price). - Get Order Info: Danh sách lệnh chờ. - Get Balance: Số dư ví khả dụng.	Sử dụng ngữ cảnh *Current_Symbol*.
FR-DESIGN-09	Khối Đặt lệnh Futures (Smart Order)	Khối thực thi "All-in-one" áp dụng cho *Current_Symbol*. - Input: Side (Long/Short), Type (Limit/Market), Price, Quantity, Leverage (1x-125x), Margin Type (Isolated/Cross).	**Logic Backend:** Kiểm tra cấu hình sàn hiện tại -> Nếu khác Input thì gửi lệnh đổi Đòn bẩy/Ký quỹ -> Sau đó mới gửi lệnh Đặt hàng.

FR-DESIGN-10	Khối Quản lý Lệnh	Các hành động dọn dẹp cho *Current_Symbol*: - Close Position: Đóng ngay vị thế hiện tại (về 0). - Cancel All Orders: Hủy toàn bộ lệnh chờ.	
FR-DESIGN-11	Lưu & Kiểm tra Mẫu (Save Template)	- Validation: Kiểm tra lỗi nối khối, cảnh báo nếu thiếu khối Sự kiện. - Save: Lưu cấu trúc sơ đồ thành "Mẫu chiến lược" vào Database.	Lưu dưới dạng JSON. Mẫu này sẽ được tái sử dụng ở Phân hệ Vận hành.
FR-DESIGN-12	Xuất mẫu chiến lược (Export JSON)	Cho phép người dùng xuất (download) cấu trúc sơ đồ khối hiện tại trên Workspace ra một tệp tin cục bộ định dạng .json.	Chuyển đổi dữ liệu từ Blockly Workspace thành chuỗi JSON và sử dụng Blob API để tải xuống từ trình duyệt.
FR-DESIGN-13	Nhập mẫu chiến lược (Import JSON)	Cho phép người dùng tải lên (upload) một tệp tin .json từ máy tính để tái hiện lại toàn bộ cấu trúc các khối lệnh và thông số trên không gian làm việc.	Kiểm tra tính hợp lệ của cấu trúc JSON trước khi nạp vào Workspace thông qua hàm *domToWorkspace* hoặc *serialization* của Blockly..

Bảng 2-2: Yêu cầu chức năng Phân hệ Thiết kế chiến lược giao dịch

2.3 Phân hệ Backtest & Live Trade

2.3.1 Chức năng Backtest

Mã Yêu cầu	Tên chức năng	Mô tả chi tiết	Ghi chú kỹ thuật
FR-RUN-01	Cấu hình Backtest	Người dùng thiết lập thông số môi trường giả lập: - Strategy Template: Chọn mẫu chiến lược - Symbol: Chọn cặp coin (VD: BTCUSDT). - Time Range: Từ ngày... Đến ngày... - Initial Capital: Vốn giả định (VD: 1000 USDT). - Fee: Phí giao dịch giả định (VD: 0.04%).	
FR-RUN-02	Thực thi mô phỏng (Simulation Engine)	Hệ thống tải dữ liệu nến lịch sử từ Database và chạy lần lượt từng cây nến qua logic của Bot. - Cơ chế: Giả lập sự kiện *On Candle Close* cho từng cây nến lịch sử. - Khớp lệnh: Giả lập khớp lệnh dựa trên giá High/Low/Close của cây nến lịch sử đó.	Backend: Cần tối ưu tốc độ xử lý để chạy nhanh nhất có thể.
FR-RUN-03	Báo cáo Hiệu suất (Performance Report)	Hiển thị kết quả tổng hợp sau khi chạy xong: - Total PnL: Tổng lãi/lỗ. - Win Rate: Tỷ lệ thắng. - Max Drawdown: Mức sụt giảm tài khoản lớn nhất.	Tính toán dựa trên danh sách lệnh đã giả lập khớp.

		- Profit Factor: Tổng thắng / Tổng thua.	
FR-RUN-04	Biểu đồ Tăng trưởng (Equity Chart)	Vẽ biểu đồ đường thể hiện sự thay đổi của số dư tài khoản theo thời gian trong quá trình Backtest.	Frontend: Sử dụng thư viện biểu đồ (VD: Lightweight Charts).

Bảng 2-3: Yêu cầu chức năng Backtest

2.3.1 Chức năng Live Trade

Mã Yêu cầu	Tên chức năng	Mô tả chi tiết	Ghi chú kỹ thuật
FR-RUN-05	Khởi tạo Bot (Create Instance)	Tạo một phiên bản chạy thực tế (Instance) từ Mẫu chiến lược: - Chọn Template (Mẫu chiến lược giao dịch). - Chọn Symbol (Cặp tiền thực tế).	Lúc này hệ thống sẽ tiêm biến *Current_Symbol* và *Template* vào bộ xử lý.
FR-RUN-06	Điều khiển Bot (Start/Stop)	- Start: Bot bắt lắng nghe sự kiện nến mới và chạy logic. - Stop: Bot ngừng lắng nghe, hủy các lệnh chờ còn lại (tùy chọn).	Trạng thái Bot: Running, Stopped, Error.
FR-RUN-07	Cơ chế chống treo (Unit Cost)	Để ngăn chặn vòng lặp vô tận (Infinite Loop) trong logic người dùng: - Mỗi khối lệnh khi chạy tốn 1 lượng "Unit". - Mỗi Session giới hạn tối đa N Unit (VD: 1000 Unit). - Nếu vượt quá -> Cưỡng chế dừng Session và báo lỗi trong log.	Bảo vệ hệ thống: Tránh việc người dùng vẽ vòng lặp While(True) làm treo Server.

FR-RUN-08	Nhật ký hoạt động (Live Logs)	Hiển thị log thời gian thực: - Thời gian kích hoạt Session. - Quyết định (Đặt lệnh hay Bỏ qua). - Số lượng Unit sử dụng	Gọi API get cho lần đầu tiên và sau đó sử dụng WebSocket đẩy log từ Backend ra Frontend.
FR-RUN-09	Giám sát Trạng thái & PnL	Dashboard hiển thị các Bot đang chạy: - Trạng thái kết nối. - Vị thế hiện tại (Đang Long hay Short). - PnL chưa chốt (Unrealized PnL) lấy từ sàn.	

Bảng 2-4: Yêu cầu chức năng Live Trade

2.4 Phân hệ theo dõi

Mã Yêu cầu	Tên chức năng	Mô tả chi tiết	Ghi chú kỹ thuật
FR-MONITOR-01	Bảng điều khiển thị trường (Market Watch)	Hiển thị danh sách các cặp tiền hỗ trợ giao dịch. Khi người dùng chọn một Symbol, Frontend sẽ yêu cầu Backend cung cấp dữ liệu nến và trạng thái Bot tương ứng.	Dữ liệu giá/biến động được Backend nhận từ sàn và đẩy về Frontend qua WebSocket
FR-MONITOR-02	Biểu đồ nến & Trực quan hóa lệnh	Hiển thị biểu đồ nến chuyên sâu. Backend cung cấp mảng dữ liệu nến lịch sử và danh sách các điểm khớp lệnh (Markers) để Frontend vẽ trực tiếp lên Chart.	Sử dụng thư viện Lightweight Charts (TradingView) để tối ưu hiển thị..
FR-MONITOR-03	Bảng quản lý Bot & Vị thế (Bottom Panel))	Hiển thị toàn bộ thông tin vận hành gồm: Danh sách Bot đang chạy, Vị thế thực tế (Size, PnL), và Nhật ký hoạt động (Logs).	Toàn bộ trạng thái này được Backend duy trì trong RAM (Bot Instance) và đồng bộ liên tục xuống Frontend
FR-MONITOR-04	Thống kê & Lịch sử giao dịch	Bảng liệt kê các lệnh đã hoàn thành với các thông số: Giá khớp, phí, và lợi nhuận thực tế.	Dữ liệu được Backend trích xuất từ Database
FR-MONITOR-05	Cảnh báo Hệ thống (System Alerts)	Hiển thị thông báo (Notification/Toast) trên giao diện khi có sự cố: - Mất kết nối API. - Lỗi thực thi logic (VD: Vòng lặp quá tải Unit Cost). - Bot bị sàn thanh lý lệnh (Liquidation Alert).	

Bảng 2-5: Yêu cầu chức năng Phân hệ theo dõi

2.5 Phân hệ Lõi: Tích hợp & Quản lý dữ liệu

Mã Yêu cầu	Tên chức năng	Mô tả chi tiết	Ghi chú kỹ thuật (PostgreSQL & Docker)
FR-CORE-01	Cầu nối API An toàn (Secure API Proxy)	Backend đóng vai trò trung gian duy nhất giao tiếp với Binance. - Frontend gửi request đến Backend (Internal API). - Backend lấy API Key (đã mã hóa) từ Database, giải mã và ký thực request gửi sang Binance.	- Storage: Key được mã hóa AES-256 và lưu trong bảng api_keys của PostgreSQL. - Security: Secret Key chỉ tồn tại ở dạng Plain-text trong RAM thời gian ngắn để ký lệnh, tuyệt đối không ghi ra Log. - Signature: Sử dụng crypto/hmac để tạo chữ ký SHA256.
FR-CORE-02	Đồng bộ dữ liệu Lai (Hybrid Data Sync)	Cơ chế kết hợp để đảm bảo dữ liệu liền mạch: 1. Real-time: Lắng nghe WebSocket để "hứng" nến mới và INSERT ngay vào Postgres. 2. On-demand: Khi truy vấn khoảng thời gian chưa có trong DB, hệ thống tự động gọi REST API tải về.	- Driver: Sử dụng GORM với driver pgx (PostgreSQL Driver) tối ưu hiệu năng. - Container Network: Backend kết nối với Database qua Docker Network (VD: dsn := "host=postgres_db user=khanh..."). - Logic: Kiểm tra db.First() trước. Nếu không có dữ liệu mới kích hoạt http.Get.
FR-CORE-03	Tự động lấp đầy (Auto-Gap Filling)	Hệ thống có tiến trình chạy ngầm (Background Worker) để phát hiện và vá lỗ hổng dữ liệu do mất mạng.	- Batch Insert: Sử dụng tính năng CreateInBatches của GORM để insert 1000 nến/lần vào Postgres hiệu quả. - Conflict Handling: Sử dụng cú pháp ON CONFLICT (symbol, open_time) DO NOTHING của

		- Tính toán Gap thời gian và tải bù dữ liệu.	PostgreSQL để tránh lỗi trùng lặp dữ liệu.
FR-CORE-04	Quản lý giới hạn tần suất (Rate Limiter)	Hệ thống tự động xếp hàng (Queue) và điều phối các yêu cầu gửi đi để tuân thủ quy định Weight của Binance.	- Algorithm: Sử dụng thuật toán Token Bucket (golang.org/x/time/rate) trong bộ nhớ ứng dụng (In-Memory). - Optimization: Đọc header x-mbx-used-weight từ response để điều chỉnh tốc độ tiêu thụ Token động.
FR-CORE-05	Lưu trữ bền vững (Persistent Storage)	Dữ liệu nến, lịch sử lệnh và cấu hình Bot được lưu trữ vĩnh viễn, không bị mất khi Restart Container.	- Docker Volume: Cấu hình volumes trong docker-compose.yml để map dữ liệu ra máy chủ vật lý (VD: - ./pgdata:/var/lib/postgresql/data). - Data Integrity: PostgreSQL đảm bảo tính toàn vẹn dữ liệu (ACID).

Bảng 2-6: Yêu cầu chức năng Phân hệ Lõi

3 Yêu cầu phi chức năng

3.1 Yêu cầu hiệu năng

Mã Yêu cầu	Tên yêu cầu	Mô tả chi tiết	Tiêu chí nghiệm thu
NFR-PERF-01	Độ trễ xử lý thực (Real-time Latency)	Thời gian trễ nội bộ (Internal Latency) từ lúc hệ thống nhận tín hiệu nến (WebSocket Event) đến khi gửi yêu cầu đặt lệnh (HTTP Request) phải được tối ưu hóa.	- Thời gian xử lý logic nội bộ: < 100ms. - Đảm bảo lệnh được gửi đi ngay lập tức sau khi tính toán xong.
NFR-PERF-02	Tốc độ Backtest	Hệ thống phải có khả năng xử lý khối lượng lớn dữ liệu nến lịch sử trong thời gian ngắn để người dùng tối ưu hóa chiến lược nhanh chóng.	- Xử lý 1 năm dữ liệu nến (khung 15 phút, khoảng ~35.000 cây nến) trong thời gian dưới 10 giây.
NFR-PERF-03	Khả năng chịu tải đồng thời	Hệ thống phải duy trì sự ổn định khi người dùng chạy nhiều Bot (Instance) cùng một lúc trên một tài khoản.	- Hỗ trợ chạy ổn định tối thiểu 5 Bot song song mà không bị suy giảm hiệu năng xử lý của từng Bot.

Bảng 3-1: Yêu cầu chức năng Phân hệ Truy cập & Cấu hình

3.2 Bảo mật

Mã Yêu cầu	Tên yêu cầu	Mô tả chi tiết	Tiêu chí nghiệm thu
NFR-SEC-01	Mã hóa Dữ liệu Nhạy cảm	API Secret Key và Access Token phải được mã hóa bằng thuật toán AES-256-GCM trước khi lưu vào Database.	Không lưu plain-text trong bất kỳ trường hợp nào.
NFR-SEC-02	Bảo mật Mật khẩu	Mật khẩu người dùng được băm (hash) bằng BCrypt/Argon2.	Không thể dịch ngược password gốc.
NFR-SEC-03	Bảo mật Kênh truyền & Reverse Proxy	Sử dụng Nginx làm Reverse Proxy để quản lý kết nối và nâng cấp bảo mật: 1. HTTP sang HTTPS: Nginx chịu trách nhiệm SSL Termination, tự động chuyển hướng mọi truy cập HTTP (cổng 80) sang HTTPS (cổng 443). 2. WS sang WSS: Nginx cấu hình nâng cấp header (Upgrade Header) để bảo mật hóa kết nối WebSocket (WSS).	- Truy cập IP/Domain qua http:// phải tự động nhảy sang https://. - Kết nối WebSocket phải qua giao thức wss:// (Secure). - Chứng chỉ SSL (VD: Let's Encrypt) được cài đặt và quản lý tại Nginx.
NFR-SEC-04	Bảo vệ Phiên (Session)	Token phiên làm việc được lưu trong HttpOnly Cookie và tự động hết hạn sau 24h.	JavaScript phía client không thể đọc được Cookie này (chống XSS).
NFR-SEC-05	Ẩn thông tin	Che dấu các ký tự nhạy cảm của API Key trên giao diện (Masking).	Hiển thị dạng ******.

Bảng 3-2: Yêu cầu phi chức năng Bảo mật

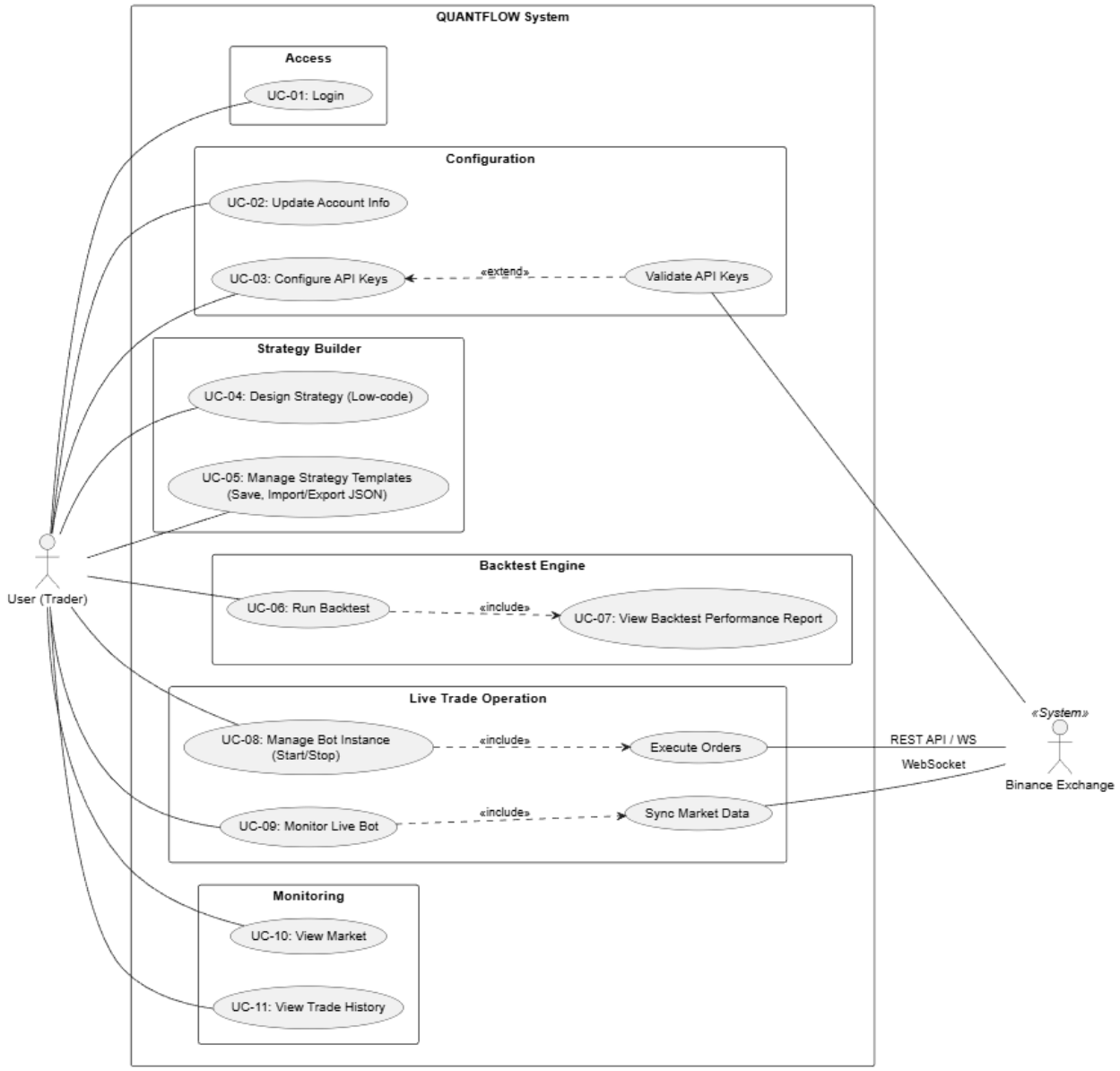
3.3 Độ tin cậy

Mã Yêu cầu	Tên yêu cầu	Mô tả chi tiết	Tiêu chí nghiệm thu
NFR-REL-01	Tự động khôi phục (Auto-recovery)	Trong trường hợp tiến trình (Process) của hệ thống bị dừng đột ngột (do lỗi phần mềm crash, panic, hoặc server khởi động lại), hệ thống phải tự động khởi chạy lại dịch vụ mà không cần sự can thiệp thủ công của con người.	- Sử dụng Docker Restart Policy (restart: always) hoặc Systemd để quản lý tiến trình. - Thời gian khôi phục dịch vụ < 30 giây sau khi sự cố xảy ra.
NFR-REL-02	Cơ chế Tự động kết nối lại (Auto-reconnect)	Khi mất kết nối WebSocket với sàn Binance (do rớt mạng hoặc sàn bảo trì), Bot phải tự động phát hiện và thực hiện kết nối lại liên tục cho đến khi thành công.	- Áp dụng thuật toán Exponential Backoff (Thử lại sau 1s, 2s, 4s, 8s...) để tránh spam API của sàn. - Không được crash khi mất mạng.
NFR-REL-03	Cô lập lỗi (Fault Isolation)	Lỗi xảy ra tại một Bot (Instance) cụ thể không được làm ảnh hưởng đến các Bot khác đang chạy song song trên cùng hệ thống.	- Sử dụng cơ chế recover() trong các Goroutine riêng biệt của Golang. - Nếu Bot A bị lỗi logic, chỉ Bot A dừng, Bot B vẫn trade bình thường.
NFR-REL-04	Tính toàn vẹn dữ liệu (Data Integrity)	Trạng thái của Bot (ví dụ: "Đang giữ lệnh Long BTC") phải được lưu xuống ổ cứng (Database/File) ngay lập tức khi thay đổi.	- Nếu Server mất điện và bật lại, Bot phải đọc Database để biết mình đang có lệnh Long hay không, thay vì bắt đầu lại từ con số 0.

Bảng 3-3: Yêu cầu phi chức năng Độ tin cậy

4 Biểu đồ use case

4.1 Biểu đồ use case tổng quan



Hình 4-1: Biểu đồ use case tổng quan

4.2 Đặc tả use case

4.2.1 UC-01: Đăng nhập (Login)

Mã Use Case	UC-01
Tên Use Case	Login
Mô tả Use Case	Cho phép người dùng truy cập vào hệ thống QuantFlow bằng cách xác thực thông tin tài khoản để sử dụng các tính năng quản lý và giao dịch.
Tác nhân	User
Mức độ ưu tiên	High
Điều kiện kích hoạt	Người dùng truy cập vào đường dẫn trang chủ của ứng dụng Web hoặc bị chuyển hướng về trang đăng nhập khi phiên làm việc hết hạn.
Tiền điều kiện	Hệ thống đang hoạt động bình thường. Tài khoản người dùng đã được khởi tạo trong cơ sở dữ liệu (Mô hình Single User/Admin).
Hậu điều kiện	Người dùng được chuyển hướng vào màn hình chính. Phiên làm việc (Session) được khởi tạo và lưu trữ an toàn.
Luồng chính	1. Hiển thị form: Hệ thống hiển thị màn hình đăng nhập với 2 trường thông tin: Tên đăng nhập (Username) và Mật khẩu (Password). 2. Nhập thông tin: Người dùng nhập Tên đăng nhập và Mật khẩu, sau đó nhấn nút "Đăng nhập". Xác thực: Hệ thống kiểm tra thông tin: 3. Mã hóa mật khẩu nhập vào và so sánh với mật khẩu đã băm (hashed) trong cơ sở dữ liệu. 4. Kiểm tra an toàn: Hệ thống kiểm tra số lần đăng nhập sai liên tiếp của tài khoản. 5. Cấp quyền: Nếu thông tin đúng và tài khoản không bị khóa, hệ thống tạo Token phiên làm việc (Session Token). 6. Lưu phiên: Hệ thống lưu Token vào HttpOnly Cookie của trình duyệt. 7. Chuyển hướng: Hệ thống chuyển hướng người dùng vào màn hình chính.
Luồng thay thế	
Luồng ngoại lệ	E1. Sai thông tin đăng nhập (<5 Lần): • Nếu Tên đăng nhập hoặc Mật khẩu không đúng. • Hệ thống hiển thị thông báo lỗi: "Tên đăng nhập hoặc mật khẩu không chính xác". • Tăng bộ đếm số lần đăng nhập sai lên 1. • Giữ người dùng tại màn hình đăng nhập. E2. Tài khoản bị khóa (Brute-force Protection): • Nếu người dùng nhập sai quá 5 lần liên tiếp. • Hệ thống khóa tính năng đăng nhập trong 15 phút. • Hiển thị thông báo: "Bạn đã nhập sai quá nhiều lần. Vui lòng thử lại sau 15 phút". E3. Lỗi kết nối Nếu không thể kết nối đến Backend hoặc Database.

	Hệ thống hiển thị thông báo “Lỗi kỹ thuật. Vui lòng thử lại sau”.
Quy tắc nghiệp vụ	1. Cơ chế khóa (Lockout Policy): Để chống tấn công dò mật khẩu (Brute-force), hệ thống bắt buộc khóa đăng nhập 15 phút sau 5 lần thử sai liên tiếp. 2. Mô hình người dùng: Hệ thống hiện tại chỉ hỗ trợ mô hình đơn người dùng (Single User), nghĩa là chỉ có một tài khoản Admin duy nhất quản trị toàn bộ hệ thống.
Yêu cầu phi chức năng	1. Bảo mật mật khẩu: Mật khẩu người dùng không được lưu dưới dạng văn bản thuần (plain-text) mà phải được băm (hash) bằng thuật toán mạnh như BCrypt hoặc Argon2. 2. Bảo vệ phiên (Session Security): Token phiên làm việc phải được lưu trong HttpOnly Cookie để ngăn chặn mã độc JavaScript phía client đọc trộm (chống lỗi XSS) và tự động hết hạn sau 24 giờ. 3. Bảo mật kênh truyền: Quá trình đăng nhập phải thực hiện qua giao thức HTTPS để mã hóa dữ liệu truyền tải.

Bảng 4-1: Đặc tả UC-01: Đăng nhập (Login)

4.2.2 UC-02: Cập nhật thông tin tài khoản (Update Account Info)

Mã Use Case	UC-02
Tên Use Case	Update Account Info
Mô tả Use Case	Cho phép người dùng thay đổi thông tin đăng nhập (Tên đăng nhập và Mật khẩu) để đảm bảo an toàn bảo mật hoặc theo nhu cầu cá nhân.
Tác nhân	User
Mức độ ưu tiên	Medium
Điều kiện kích hoạt	Người dùng chọn "Tài khoản" từ giao diện Cài đặt
Tiền điều kiện	Người dùng đã đăng nhập thành công vào hệ thống.
Hậu điều kiện	Thông tin (Tên đăng nhập/Mật khẩu) được cập nhật mới trong Cơ sở dữ liệu. Người dùng bị đăng xuất khỏi hệ thống và phải đăng nhập lại.
Luồng chính	1. Truy cập chức năng: Trên màn hình chính, người dùng nhấn vào "Cấu hình" phía dưới cùng của Sidebar bên trái. Hệ thống hiển thị Dialog cài đặt (gồm Sidebar trái và vùng nội dung phải), sau đó người dùng chọn vào "Tài khoản" trong sidebar. 2. Hiển thị biểu mẫu: Hệ thống hiển thị biểu mẫu cập nhật gồm các trường: • Tên đăng nhập mới (New Username). • Mật khẩu mới (New Password). • Nhập lại mật khẩu mới (Confirm Password). • Mật khẩu hiện tại (Current Password - Bắt buộc). 3. Nhập thông tin: Người dùng nhập các thông tin muốn thay đổi. • Lưu ý: Người dùng bắt buộc phải nhập đúng "Mật khẩu hiện tại" để xác thực quyền thay đổi. 4. Gửi yêu cầu: Người dùng nhấn nút "Lưu thay đổi" (Save Changes). 5. Hệ thống kiểm tra (Validation): • Kiểm tra "Mật khẩu hiện tại" có khớp với mật khẩu trong Database không. • Kiểm tra độ trùng khớp giữa "Mật khẩu mới" và "Nhập lại mật khẩu mới". • Kiểm tra độ mạnh của mật khẩu mới (nếu có thay đổi). 6. Cập nhật dữ liệu: • Nếu thỏa mãn, hệ thống tiến hành băm (hash) mật khẩu mới (nếu có). • Lưu Tên đăng nhập mới và Mật khẩu đã băm vào Database. 7. Xử lý hậu kỳ (Force Logout): Hệ thống tự động hủy phiên làm việc hiện tại (xóa Session/Cookie). 8. Thông báo & Chuyển hướng: Hệ thống hiển thị thông báo "Cập nhật thành công. Vui lòng đăng nhập lại" và chuyển hướng người dùng về màn hình Đăng nhập (Login).
Luồng thay thế	A1. Chỉ đổi Tên đăng nhập (giữ nguyên Mật khẩu): Người dùng chỉ nhập "Tên đăng nhập mới" và để trống các ô "Mật khẩu mới".

	Hệ thống chỉ cập nhật Username, giữ nguyên Password cũ. Vẫn yêu cầu nhập "Mật khẩu hiện tại" để xác nhận. A2. Chỉ đổi Mật khẩu (giữ nguyên Tên đăng nhập): Người dùng giữ nguyên "Tên đăng nhập" cũ, chỉ nhập các trường liên quan đến Mật khẩu mới.
Luồng ngoại lệ	E1. Sai mật khẩu hiện tại: • Hệ thống kiểm tra thấy "Mật khẩu hiện tại" không đúng. • Hiển thị lỗi: "Mật khẩu hiện tại không chính xác. Vui lòng thử lại." • Không thực hiện cập nhật DB. E2. Xác nhận mật khẩu không khớp: • Hệ thống thấy "Mật khẩu mới" và "Nhập lại mật khẩu mới" khác nhau. • Hiển thị lỗi: "Mật khẩu xác nhận không trùng khớp." E3. Tên đăng nhập mới trùng với tên cũ: • (Tùy chọn cảnh báo): Hệ thống thông báo "Tên đăng nhập mới giống tên hiện tại" (nếu cần thiết, tuy nhiên không phải lỗi nghiêm trọng).
Quy tắc nghiệp vụ	1. Bắt buộc xác thực: Mọi thay đổi liên quan đến thông tin nhạy cảm (Credentials) đều bắt buộc người dùng phải cung cấp mật khẩu hiện tại (Current Password) để chống chiếm đoạt phiên làm việc (Session Hijacking). 2. Cơ chế đăng xuất cưỡng chế: Sau khi thay đổi thành công Username hoặc Password, hệ thống bắt buộc phải đăng xuất người dùng (Logout). Điều này đảm bảo Token cũ bị vô hiệu hóa và người dùng phải tạo Session mới với thông tin xác thực mới.
Yêu cầu phi chức năng	1. Bảo mật dữ liệu: Mật khẩu mới khi lưu xuống Database phải được mã hóa (Hash) bằng thuật toán BCrypt, tuyệt đối không lưu dạng văn bản thuần (plain-text). 2. Thông báo rõ ràng: Thông báo yêu cầu đăng nhập lại cần hiển thị rõ ràng để người dùng không bị bối rối khi đột ngột bị thoát khỏi hệ thống.

Bảng 4-2: Đặc tả UC-02: Cập nhật thông tin tài khoản (Update Account Info)

4.2.3 UC-03: Cấu hình API Sàn (Configure API Keys)

Mã Use Case	UC-03
Tên Use Case	Configure API Keys
Mô tả Use Case	Cho phép người dùng nhập và lưu trữ an toàn cặp khóa API (API Key & Secret Key) của sàn Binance, cấp quyền cho hệ thống thay mặt người dùng thực hiện các thao tác lấy dữ liệu và đặt lệnh giao dịch.
Tác nhân	User
Mức độ ưu tiên	High
Điều kiện kích hoạt	Người dùng chọn "Kết nối sàn" từ giao diện Cài đặt.
Tiền điều kiện	Người dùng đã đăng nhập thành công vào hệ thống. Người dùng đã có tài khoản Binance và tạo thành công cặp khóa API (đã bật quyền Futures Trading).
Hậu điều kiện	Cặp khóa API được lưu vào Cơ sở dữ liệu (trong đó Secret Key được mã hóa). Hệ thống chuyển trạng thái kết nối sang "Đã kết nối" (Connected).
Luồng chính	1. Truy cập chức năng: Trên màn hình chính, người dùng nhấn vào “Cấu hình” phía dưới cùng của Sidebar bên trái. Hệ thống hiển thị Dialog cài đặt (gồm Sidebar trái và vùng nội dung phải), sau đó người dùng chọn vào “Kết nối sàn” trong sidebar. 2. Hiển thị biểu mẫu: Hệ thống hiển thị biểu mẫu cấu hình gồm các trường: • Chọn sàn giao dịch (Dropdown - Mặc định: Binance). • API Key (Input text). • Secret Key (Input password - Che dấu ký tự). • Trạng thái hiện tại: Chưa kết nối / Đã kết nối. 3. Nhập thông tin: Người dùng thực hiện dán (paste) chuỗi API Key và Secret Key lấy từ sàn Binance vào các ô tương ứng. 4. Gửi yêu cầu: Người dùng nhấn nút "Lưu cấu hình" (Save Configuration). 5. Hệ thống kiểm tra (Validation): • Kiểm tra định dạng chuỗi Key (độ dài, ký tự đặc biệt). • Gửi yêu cầu kiểm tra (Ping/Account Info) đến Binance API để xác thực tính đúng đắn và quyền hạn của Key. 6. Lưu trữ dữ liệu: • Nếu xác thực thành công từ phía Binance. • Hệ thống thực hiện mã hóa Secret Key (AES-256). • Lưu thông tin vào Database (API Key lưu dạng rõ, Secret Key lưu dạng mã hóa). 7. Thông báo & Cập nhật: • Hệ thống hiển thị thông báo "Kết nối sàn thành công". • Cập nhật trạng thái trên giao diện thành "Đã kết nối" (màu xanh).
Luồng thay thế	A1. Cập nhật lại API Key: • Người dùng muốn thay đổi cặp khóa cũ.

	• Khi mở form, ô API Key hiển thị một phần (ví dụ: ******ABCD), ô Secret Key để trống. • Người dùng nhập đè Key mới vào và thực hiện quy trình lưu như Luồng chính.
Luồng ngoại lệ	E1. Kết nối bị từ chối bởi Sàn: • Hệ thống gửi thử nghiệm thất bại (do sai Key, sai IP whitelist, hoặc chưa bật quyền Futures). • Hiển thị thông báo lỗi chi tiết từ sàn: "API Key không hợp lệ hoặc thiếu quyền hạn". • Không lưu dữ liệu vào Database. E2. Định dạng không hợp lệ: • Người dùng nhập thiếu ký tự hoặc có khoảng trắng thừa. • Hệ thống báo lỗi ngay tại trường nhập liệu: "Vui lòng kiểm tra lại định dạng API Key".
Quy tắc nghiệp vụ	1. Bảo mật Secret Key (Write-Only): Hệ thống chỉ cho phép nhập Secret Key một chiều. Khi người dùng mở lại form cấu hình, ô Secret Key phải luôn để trống (hoặc hiển thị placeholder ********), tuyệt đối không được gửi trả lại Secret Key từ Server xuống Client để hiển thị. 2. Mã hóa lưu trữ: Bắt buộc phải mã hóa Secret Key bằng thuật toán đối xứng (Symmetric Encryption) trước khi lưu vào đĩa cứng để đảm bảo an toàn nếu Database bị lộ.
Yêu cầu phi chức năng	1. Chuẩn mã hóa: Sử dụng thuật toán AES-256-GCM để mã hóa Secret Key. 2. Che dấu thông tin (Masking): Trên giao diện, API Key chỉ được hiển thị 4 ký tự cuối, các ký tự đầu phải được che bằng dấu sao (*).

Bảng 4-3: Đặc tả UC-03: Cấu hình API Sàn (Configure API Keys)

4.2.4 UC-04: Thiết kế chiến lược (Design Strategy)

Mã Use Case	UC-04
Tên Use Case	Design Strategy
Mô tả Use Case	Cung cấp môi trường lập trình trực quan (Drag-and-drop), cho phép người dùng lắp ghép các khối lệnh (Sự kiện, Logic, Chỉ báo, Giao dịch) để xây dựng kịch bản hoạt động cho Bot mà không cần viết code thủ công.
Tác nhân	User
Mức độ ưu tiên	High
Điều kiện kích hoạt	Người dùng chọn “Tạo nhanh” trên Sidebar chính của ứng dụng. Người dùng tạo mới hoặc chỉnh sửa chiến lược trong màn hình Chiến lược (UC-05).
Tiền điều kiện	Người dùng đã đăng nhập thành công vào hệ thống.
Hậu điều kiện	Một bản ghi chiến lược mới được tạo trong Cơ sở dữ liệu. Dữ liệu logic của chiến lược được lưu trữ dưới định dạng chuẩn JSON.
Luồng chính	1. Truy cập không gian làm việc: Từ màn hình danh sách chiến lược, người dùng nhấn nút "Tạo mới" hoặc người dùng nhấn nút “Tạo nhanh” trên Sidebar. Hệ thống chuyển sang trang Chỉnh sửa. 2. Khởi tạo giao diện: Hệ thống hiển thị: • Toolbox (Thanh công cụ - Bên trái): Chứa các danh mục khối lệnh (Sự kiện, Logic, Toán học, Chỉ báo, Giao dịch, Biến). • Workspace (Vùng thao tác - Giữa): Vùng trống vô hạn để kéo thả. • Control Panel (Thanh điều khiển - Bên phải/Trên): Các nút Zoom, Undo/Redo, Canh chỉnh, Lưu. 3. Định nghĩa Sự kiện (Bắt buộc): • Người dùng kéo khối "Sự kiện" (Event Trigger) từ Toolbox vào Workspace. • Người dùng cấu hình tham số: Khung thời gian (VD: 1m, 15m, 1h) và Loại kích hoạt (Khi đóng nến / Khi mở nến). 4. Xây dựng Logic tính toán: • Người dùng kéo các khối "Chỉ báo kỹ thuật" (VD: RSI, EMA). Các khối này tự động nhận diện dữ liệu từ cặp tiền hiện tại (Context-aware). • Người dùng kết hợp với các khối "Logic/Toán học" (So sánh >, <, =, If/Else) để tạo điều kiện vào lệnh. 5. Định nghĩa Hành động Giao dịch: • Khi điều kiện thỏa mãn, người dùng kéo khối "Đặt lệnh Futures" (Smart Order) vào luồng. • Cấu hình tham số lệnh: Mua/Bán (Long/Short), Loại lệnh (Limit/Market), Số lượng (Quantity), Đòn bẩy (Leverage). 6. Kiểm tra & Lưu trữ: • Người dùng nhấn nút "Lưu chiến lược" (Save). • Hệ thống yêu cầu nhập "Tên chiến lược" (nếu là tạo mới). 8. Hệ thống xác thực (Validation):

	• Kiểm tra sự tồn tại của khối Sự kiện (Event Trigger). • Kiểm tra các khối lệnh có được nối liền mạch hay không (không có khối rời rạc quan trọng). 9. Lưu xuống Database: • Hệ thống chuyển đổi toàn bộ sơ đồ khối thành chuỗi JSON. • Lưu vào cơ sở dữ liệu và hiển thị thông báo "Lưu chiến lược thành công".
Luồng thay thế	A1. Chỉnh sửa chiến lược có sẵn: • Người dùng chọn một chiến lược từ danh sách và nhấn "Sửa". • Hệ thống tải dữ liệu JSON từ Database, tái tạo lại sơ đồ khối y hệt trạng thái lần lưu trước đó lên Workspace. • Người dùng tiếp tục chỉnh sửa và nhấn Lưu. A2. Sử dụng Biến nhớ (Variable): • Người dùng tạo biến để lưu giá trị tạm thời. • Người dùng chọn phạm vi biến (Scope): • Biến Phiên (Session): Tự hủy khi hết nến. • Biến Vòng đời (Lifecycle): Giữ giá trị vĩnh viễn (ví dụ: đếm số lần thắng liên tiếp).
Luồng ngoại lệ	E1. Thiếu khối sự kiện khởi đầu: • Người dùng nhấn Lưu nhưng trên Workspace không có khối "Sự kiện". • Hệ thống báo lỗi: "Chiến lược phải bắt đầu bằng khối Sự kiện (Event Trigger)". • Hủy thao tác lưu. E2. Lỗi kết nối khối (Disconnected Blocks): • Các khối logic quan trọng không được gắn vào luồng chính (nằm lơ lửng). • Hệ thống cảnh báo: "Phát hiện khối lệnh chưa được kết nối. Vẫn lưu?". E3. Dữ liệu đầu vào không hợp lệ: • Người dùng nhập chữ cái vào trường nhập số (ví dụ: nhập "abc" vào ô Đòn bẩy). • Blockly tự động từ chối hoặc báo đỏ ngay tại ô nhập liệu.
Quy tắc nghiệp vụ	1. Context-Aware (Ngữ cảnh hóa): Các khối chỉ báo (RSI, MACD) và khối đặt lệnh không yêu cầu người dùng chọn cặp coin (Symbol) trong lúc thiết kế. Chúng sẽ tự động lấy dữ liệu của cặp coin đang chạy thực tế (hoặc cặp coin được chọn lúc Backtest). 2. Logic một chiều: Luồng xử lý luôn bắt đầu từ khối Sự kiện và chạy tuần tự xuống dưới. Không hỗ trợ chạy ngược hoặc chạy song song đa luồng trong một chiến lược đơn lẻ. 3. Giới hạn tài nguyên: Để tránh làm treo trình duyệt, hệ thống giới hạn số lượng khối tối đa trên một Workspace (ví dụ: 1024 khối).
Yêu cầu phi chức năng	1. Thư viện lõi: Bắt buộc sử dụng Google Blockly để render giao diện.

	2. Định dạng lưu trữ: Dữ liệu đầu ra phải là JSON String chuẩn, đảm bảo có thể tái sử dụng cho module Backtest Engine (UC-06) và Live Trade (UC-08) mà không cần parse lại logic. 3. Trải nghiệm người dùng: Hỗ trợ các phím tắt cơ bản: Ctrl+Z (Undo), Ctrl+C/V (Copy/Paste khối), Scroll chuột để Zoom in/out.

Bảng 4-4: Đặc tả UC-04: Thiết kế chiến lược (Design Strategy)

4.2.5 UC-05: Quản lý mẫu chiến lược (Manage Strategy Templates)

Mã Use Case	UC-05
Tên Use Case	Manage Strategy Templates
Mô tả Use Case	Cho phép người dùng lưu trữ các chiến lược đã thiết kế vào cơ sở dữ liệu để tái sử dụng, đồng thời hỗ trợ Xuất (Export) ra file hoặc Nhập (Import) từ file để sao lưu cục bộ hoặc chia sẻ giữa các thiết bị.
Tác nhân	User
Mức độ ưu tiên	Medium
Điều kiện kích hoạt	Người dùng chọn menu "Chiến lược" từ Sidebar chính của ứng dụng.
Tiền điều kiện	Người dùng đã đăng nhập thành công. Đã có ít nhất một chiến lược được lưu (để thực hiện các hành động Xem/Xóa/Xuất).
Hậu điều kiện	Danh sách chiến lược được cập nhật (sau khi Xóa hoặc Nhập). File JSON được tải xuống máy (sau khi Xuất).
Luồng chính	1. Truy cập trang quản lý: Người dùng chọn "Chiến lược" từ Sidebar chính của ứng dụng. Hệ thống hiển thị trang Quản lý chiến lược. 2. Hiển thị danh sách: Hệ thống liệt kê các chiến lược dưới dạng bảng (Table), bao gồm các thông tin: • Tên chiến lược. • Ngày tạo. • Ngày cập nhật cuối. • Trạng thái (Hợp lệ / Nháp). • Hành động (Sửa, Xóa, Xuất). 3. Điều hướng chỉnh sửa: • Người dùng nhấn nút "Sửa" trên một dòng chiến lược. • Hệ thống chuyển hướng sang trang Chỉnh sửa (UC-04) và tải dữ liệu của chiến lược đó lên. 4. Điều hướng tạo mới: • Người dùng nhấn nút "Tạo mới" (Create New) ở góc trên màn hình. • Hệ thống chuyển hướng sang trang Chỉnh sửa (UC-04) với không gian làm việc trống.
Luồng thay thế	
Luồng ngoại lệ	A1. Nhập chiến lược từ file (Import JSON) ngay tại trang Quản lý • Người dùng nhấn nút "Nhập từ file" (Import) trên thanh công cụ của trang Quản lý. • Hệ thống hiển thị Dialog yêu cầu chọn file. • Người dùng tải lên tệp .json từ máy tính. • Hệ thống kiểm tra cấu trúc file (Validate JSON). • Lưu ý: Nếu hợp lệ, hệ thống tự động tạo một bản ghi chiến lược mới trong Database với nội dung từ file đó. • Hệ thống làm mới (Refresh) danh sách để hiển thị chiến lược vừa nhập. A2. Xuất chiến lược ra file (Export JSON) từ danh sách

	• Tại dòng của chiến lược mong muốn, người dùng nhấn menu ngữ cảnh (dấu 3 chấm) hoặc icon "Xuất" (Export). • Hệ thống lấy dữ liệu JSON của chiến lược đó từ Database (Backend). • Trình duyệt tự động tải xuống tệp tin .json (Ví dụ: chien-luoc-ema.json). A3. Xóa chiến lược • Người dùng nhấn nút "Xóa" (Delete) trên một dòng chiến lược. • Hệ thống hiển thị hộp thoại xác nhận: "Bạn có chắc chắn muốn xóa chiến lược này không?". • Người dùng chọn "Đồng ý". • Hệ thống cập nhật trạng thái bản ghi trong Database và cập nhật lại danh sách hiển thị.
Quy tắc nghiệp vụ	1. Tính toàn vẹn dữ liệu: Khi xóa một chiến lược, hệ thống phải kiểm tra xem chiến lược đó có đang được sử dụng bởi một Bot đang chạy hay không. Nếu đang chạy, hệ thống phải ngăn chặn việc xóa và báo lỗi "Chiến lược đang được sử dụng bởi Bot [ID]. Vui lòng dừng Bot trước khi xóa". 2. Cảnh báo khi sửa: Khi người dùng mở Editor để sửa một chiến lược đang được sử dụng bởi ít nhất một Bot đang chạy (trạng thái Running), hệ thống phải hiển thị cảnh báo (Banner/Toast): "Chiến lược này đang được sử dụng bởi Bot đang chạy. Các thay đổi của bạn sẽ chỉ áp dụng cho các phiên chạy mới."
Yêu cầu phi chức năng	1. Phân trang (Pagination): Nếu người dùng có quá nhiều chiến lược (ví dụ: > 20), danh sách phải hỗ trợ phân trang để đảm bảo tốc độ tải trang nhanh. 2. Tìm kiếm (Search): Trang quản lý phải có thanh tìm kiếm để lọc nhanh chiến lược theo tên. 3. Lưu trữ phiên bản: Khi Bot chạy, nó lưu lại strategy_id và version mà nó đang dùng (ví dụ: v1). Khi người dùng sửa và lưu, bản ghi trong database tăng lên v2. Điều này giúp truy vết lịch sử (Audit Log) để biết tại sao Bot lại vào lệnh đó (do logic v1 hay v2).

Bảng 4-5: Đặc tả UC-05: Quản lý mẫu chiến lược (Manage Strategy Templates)

4.2.6 UC-06: Thực thi Backtest (Run Backtest)

Mã Use Case	UC-06
Tên Use Case	Run Backtest
Mô tả Use Case	Cho phép người dùng cấu hình và chạy mô phỏng chiến lược giao dịch.
Tác nhân	User
Mức độ ưu tiên	Low
Điều kiện kích hoạt	Người dùng chọn Tab "Backtest" tại màn hình "Giao dịch".
Tiền điều kiện	Người dùng đã đăng nhập thành công. Đã có ít nhất một Chiến lược (Strategy) khả dụng trong hệ thống để thực hiện backtest.
Hậu điều kiện	Hệ thống chuyển sang giao diện Hiệu suất Backtest. Kết quả thống kê (PnL, Winrate) và biểu đồ biến động tài sản (Equity Mini-Chart) được hiển thị.
Luồng chính	1. Truy cập chức năng: • Người dùng truy cập mục "Giao dịch" trên Sidebar. • Tại khu vực điều khiển phía dưới biểu đồ chính (Main Chart), người dùng chọn Tab "Backtest". 2. Cấu hình thông số: Hệ thống hiển thị Form nhập liệu bao gồm các trường: • Chọn Chiến lược: Dropdown danh sách các chiến lược cá nhân. • Symbol: Cặp tiền muốn test (VD: BTCUSDT) - Mặc định lấy theo cặp tiền đang xem trên Chart. • Timeframe: Khung thời gian nến (Dropdown: 1m, 15m, 1h, 4h, 1D...). • Time Range: Chọn "Thời gian bắt đầu" và "Thời gian kết thúc" (Date Picker). • Initial Capital: Nhập vốn giả lập (VD: 1000 USDT). • Fee: Nhập phí giao dịch giả lập mỗi lệnh (VD: 0.04%). • Unit: Nhập giới hạn Unit cho mỗi phiên chạy mặc định là 1000. 3. Kích hoạt: Người dùng nhấn nút "Bắt đầu" (Start Backtest). 4. Xử lý & Phản hồi tiến trình (Processing): • Hệ thống khóa nút "Bắt đầu" (chuyển sang trạng thái Disabled hoặc Loading). • Hiển thị Thanh tiến trình (Progress Bar) chạy từ 0% đến 100% để thể hiện trạng thái tải dữ liệu và tính toán mô phỏng của Backend. • Backend: Tải dữ liệu nến lịch sử -> Chạy logic chiến lược -> Giả lập khớp lệnh -> Tổng hợp kết quả. 5. Chuyển hướng (Transition): • Khi thanh tiến trình đạt 100%, hệ thống tự động chuyển vùng hiển thị (hoặc chuyển trang) sang giao diện "Backtest Performance". 6. Hiển thị kết quả (Result Display): Hệ thống hiển thị báo cáo gồm: • Thống kê tổng quan: Total PnL (Lãi/Lỗ), Win Rate (Tỷ lệ thắng), Max Drawdown (Sụt giảm tối đa), Profit Factor.

	• Biểu đồ tài sản (Equity Mini-Chart): Một biểu đồ đường nhỏ (Sparkline hoặc Mini-chart) thể hiện số tiền dao động theo thời gian từ lúc bắt đầu đến lúc kết thúc. • Nút thao tác: "Chạy lại" (Reset/Rerun) hoặc "Quay về cấu hình".
Luồng thay thế	A1. Hủy bỏ giữa chừng: • Trong khi thanh Process đang chạy, người dùng nhấn nút "Hủy" (Cancel/Stop). • Hệ thống dừng tiến trình, thanh Process biến mất, giao diện quay lại trạng thái nhập liệu (Bước 2). A2. Không đủ dữ liệu nến: • Chọn Time Range sẽ giới hạn tới dữ liệu mà Database hiện có. • Không cho người dùng chọn ngoài Time Range mà Database không có dữ liệu nến.
Luồng ngoại lệ	
Quy tắc nghiệp vụ	1. Giả lập khớp lệnh: Hệ thống giả định lệnh Market khớp ngay tại giá Open của nến tiếp theo. Lệnh Limit khớp dựa trên giá High/Low của nến trong khung thời gian đã chọn. 2. Tối ưu hiển thị: Biểu đồ Mini-chart chỉ cần hiển thị đường cong tài sản (Equity Curve), không cần hiển thị nến chi tiết để giúp người dùng nắm bắt xu hướng lợi nhuận nhanh chóng.
Yêu cầu phi chức năng	Hiệu năng phản hồi: Thanh tiến trình phải cập nhật mượt mà (Real-time update) thông qua cơ chế WebSocket hoặc Polling để người dùng không cảm thấy ứng dụng bị treo. Tốc độ: Thời gian xử lý cho 1 năm dữ liệu (Timeframe 15m) không được vượt quá 10 giây (theo NFR-PERF-02).

Bảng 4-6: Đặc tả UC-06: Thực thi Backtest (Run Backtest)

4.2.7 UC-08: Khởi tạo & Điều khiển Bot (Manage Bot Instance)

Mã Use Case	UC-08
Tên Use Case	Manage Bot Instance
Mô tả Use Case	Cho phép người dùng tạo mới một phiên bản thực thi (Instance) từ mẫu chiến lược có sẵn, và kiểm soát vòng đời của Bot (Chạy/Dừng) trên thị trường thật.
Tác nhân	User
Mức độ ưu tiên	High
Điều kiện kích hoạt	Người dùng chọn Tab "Bot" tại màn hình "Giao dịch".
Tiền điều kiện	Người dùng đã đăng nhập thành công. Đã cấu hình API Key (UC-03) và có ít nhất một Chiến lược (UC-04). Tài khoản Binance Futures có số dư khả dụng.
Hậu điều kiện	Bot mới được tạo và hiển thị trong danh sách quản lý. Bot bắt đầu chạy ngầm, và thực hiện giao dịch.
Luồng chính	1. Truy cập không gian vận hành: • Từ Sidebar, người dùng vào trang "Giao dịch". • Tại khu vực Panel bên dưới biểu đồ (Chart), người dùng chọn Tab "Bot". • Hệ thống hiển thị giao diện quản lý Bot gồm: Nút "Tạo Bot mới" và Danh sách Bot (dạng bảng hoặc cây). 2. Khởi tạo Bot: Người dùng nhấn nút "Tạo Bot mới" (Create New Bot). Hệ thống hiển thị Dialog cấu hình (Pop-up), yêu cầu nhập: • Tên Bot (Bot Name). • Chiến lược (Strategy Template). • Cặp tiền (Symbol): Mặc định lấy theo cặp tiền đang xem trên Chart. 3. Xác nhận & Chạy: • Người dùng kiểm tra thông tin và nhấn "Khởi chạy" (Start). • Hệ thống thực hiện Snapshot chiến lược (Deep Copy logic) và khởi tạo tiến trình ngầm. 4. Hiển thị & Giám sát (Listview Tree): Dialog đóng lại, Bot mới xuất hiện ngay trong danh sách tại Tab "Bot". Người dùng tương tác với Listview Tree: Cấp 1 (Bot Row): Hiển thị tóm tắt: Tên Bot | Cặp tiền | Trạng thái (Running/Stopped) | Tổng PnL | Nút điều khiển (Dừng/Sửa/Xóa). Cấp 2 (Expand Details): Khi người dùng nhấn mở rộng (icon + hoặc mũi tên) một dòng Bot, hệ thống hiển thị chi tiết bên dưới: • Vị thế hiện tại: (VD: Long BTC, Entry: 50000, Size: 0.1, PnL: +5$). • Các lệnh chờ (Open Orders): List các lệnh Limit/Stop đang treo.
Luồng thay thế	A1. Dừng Bot (Stop): • Tại dòng của Bot đang chạy trong danh sách, người dùng nhấn nút "Dừng" (Stop Icon).

	• Hệ thống hỏi xác nhận: "Dừng và Đóng vị thế" hay "Chỉ dừng Bot". • Người dùng chọn phương án. • Trạng thái Bot chuyển sang màu xám/đỏ (Stopped). A2. Xem chi tiết lệnh (View Orders): • Người dùng mở rộng (Expand) một Bot trong danh sách. • Hệ thống cập nhật thời gian thực (Real-time) các thông số PnL và trạng thái lệnh thông qua WebSocket.
Luồng ngoại lệ	E1. Đặt lệnh thất bại (Lỗi số dư / Ký quỹ sàn): • Khi logic chiến lược chạm điều kiện Đặt lệnh, hệ thống đẩy lệnh lên Binance nhưng tài khoản Futures không đủ số dư cho Quantity quy định, hoặc sàn từ chối do Leverage không hợp lệ. • Hệ thống thêm lỗi tại logs của Bot: "Lỗi đặt lệnh: Không đủ số dư khả dụng". Nhưng vẫn giữ Bot chạy bình thường. E2. Lỗi kết nối WebSocket: Bot bị mất luồng dữ liệu giá thị trường. • Trạng thái trên Listview chuyển sang màu vàng (Reconnecting). Hệ thống tự động thử kết nối lại ngầm (Auto-reconnect).
Quy tắc nghiệp vụ	1. Ủy quyền cấu hình cho Logic (Logic-driven Configuration): * Tại thời điểm thực thi lệnh, Backend sẽ tự động đọc tham số Leverage và Margin Type (Isolated/Cross) từ khối "Smart Order" trong JSON của chiến lược. • Trước khi đặt lệnh mua/bán thực tế, hệ thống tự động gửi REST API lên Binance để chuyển đổi Đòn bẩy và Kiểu Ký Quỹ của tài khoản cho khớp với thiết lập của khối (theo đúng FR-DESIGN-09). Người dùng không cần cấp "Vốn giả định" cho Bot. 2. Snapshot Versioning: Bot luôn hoạt động trên bản sao của chiến lược tại thời điểm tạo. Sửa chiến lược gốc không ảnh hưởng Bot đang chạy. 3. Cập nhật thời gian thực (Real-time Sync): Lợi nhuận PnL và trạng thái lệnh trên Listview Tree phải được Backend đồng bộ liên tục qua WebSocket xuống giao diện Frontend để tương xứng với nhịp đập của Chart giá ở phía trên.
Yêu cầu phi chức năng	1. Giao diện Compact: Thiết kế Tree View phải tận dụng chiều cao khiêm tốn của Panel dưới Chart, hỗ trợ cuộn dọc mượt mà. 2. Quản lý đa luồng (Concurrency): Hệ thống phải duy trì ổn định khi chạy nhiều Bot (NFR-PERF-03). Việc một Bot đang xử lý API request đổi đòn bẩy/đặt lệnh tuyệt đối không được chặn luồng nhận dữ liệu giá của các Bot khác.

Bảng 4-7: Đặc tả UC-08: Khởi tạo & Điều khiển Bot (Manage Bot Instance)

4.2.8 UC-09: Giám sát Bot đang chạy (Monitor Live Bot)

Mã Use Case	UC-09
Tên Use Case	Monitor Live Bot
Mô tả Use Case	Cho phép người dùng theo dõi trạng thái hoạt động, thông số lợi nhuận (PnL), vị thế hiện tại và nhật ký hệ thống (Live Logs) của các Bot đang chạy theo thời gian thực thông qua Listview Tree và màn hình Console thu nhỏ.
Tác nhân	User
Mức độ ưu tiên	High
Điều kiện kích hoạt	Người dùng đang ở màn hình "Giao dịch", chọn Tab "Bot" phía dưới biểu đồ giá
Tiền điều kiện	Người dùng đã đăng nhập thành công. Đã có ít nhất một Bot (Instance) được khởi tạo và tồn tại trong danh sách.
Hậu điều kiện	Người dùng nắm bắt được toàn cảnh tình trạng vận hành của Bot (Lãi/lỗ, lệnh đang treo, tiến trình logic) để đưa ra quyết định phù hợp.
Luồng chính	1. Hiển thị tổng quan (Cấp 1 - Bot Row): Hệ thống hiển thị danh sách các Bot tại Tab "Bot" dưới dạng Listview Tree. Tại cấp 1, hệ thống cung cấp các thông tin tóm tắt: Tên Bot, Cặp tiền, Trạng thái hoạt động (Running/Stopped/Error), và Tổng PnL. Thông tin PnL chưa chốt (Unrealized PnL) và trạng thái được lấy từ sàn và cập nhật liên tục lên Dashboard. 2. Xem chi tiết Vị thế & Lệnh chờ (Cấp 2 - Expand Details): Người dùng click vào icon mở rộng (dấu + hoặc mũi tên) ở đầu mỗi dòng Bot. Hệ thống mở rộng Cấp 2 hiển thị chi tiết: Vị thế hiện tại (Đang Long hay Short, Size, Entry, PnL) và danh sách các lệnh chờ (Open Orders) đang treo. 3. Tương tác chuột phải (Context Menu): Để xem tiến trình xử lý chi tiết, người dùng click chuột phải vào dòng của một Bot bất kỳ trong Listview Tree. Hệ thống hiển thị một Context Menu với các tùy chọn tương tác. 4. Mở giao diện Live Logs: Người dùng chọn mục "Logs" từ Context Menu. Hệ thống sẽ mở ra một cửa sổ Popup (cửa sổ nổi) có giao diện nền tối, mô phỏng một màn hình dòng lệnh (Console App). 5. Đồng bộ và hiển thị Logs: Hệ thống gọi API get để tải lịch sử log hoạt động cho lần mở đầu tiên. Sau đó, thông qua kết nối WebSocket, hệ thống đẩy liên tục các dòng log từ Backend ra Frontend theo thời gian thực. Các dòng log sẽ hiển thị chi tiết: Thời gian kích hoạt Session, Quyết định (Đặt lệnh hay Bỏ qua), và Số lượng Unit đã sử dụng.
Luồng thay thế	A1. Tự động cuộn Logs (Auto-scroll): Khi Console Popup đang mở và có dòng log mới bắn ra, giao diện tự động cuộn (scroll) xuống dòng dưới cùng để người dùng luôn nhìn thấy luồng xử lý mới nhất. A2. Đóng Console Logs: Người dùng nhấn dấu "X" trên góc Popup hoặc click chuột ra ngoài vùng nền tối để đóng cửa sổ Logs, quay lại giao diện Listview Tree bình thường.

Luồng ngoại lệ	E1. Mất kết nối đồng bộ: Trong quá trình giám sát, nếu mất kết nối WebSocket , hệ thống sẽ chuyển trạng thái trên Listview Tree sang màu vàng (Reconnecting). Màn hình Console Logs sẽ hiện dòng chữ màu vàng cảnh báo "Mất kết nối. Đang thử kết nối lại..." và tạm ngừng hiển thị log mới.
Quy tắc nghiệp vụ	. Đồng bộ thời gian thực (Real-time Sync): Lợi nhuận PnL, vị thế, và Logs phải được Backend đồng bộ liên tục qua WebSocket xuống giao diện Frontend để tương xứng với nhịp đập của Chart giá ở phía trên. 2. Che dấu thông tin nhạy cảm: Secret Key chỉ tồn tại ở dạng Plain-text trong RAM thời gian ngắn để ký lệnh, tuyệt đối không được xuất ra (ghi ra) màn hình Log hay Database
Yêu cầu phi chức năng	1. Hiệu năng hiển thị UI: Cửa sổ Popup Console phải được tối ưu DOM (ví dụ sử dụng Virtual Scroll hoặc giới hạn hiển thị 1000 dòng log gần nhất) để trình duyệt không bị đơ/giật lag khi Bot hoạt động liên tục với tần suất cao. 2. Giám sát đa nhiệm: Việc mở Popup Logs của một Bot hoàn toàn không được làm gián đoạn tiến trình đồng bộ giá (PnL) trên Listview Tree của các Bot khác.

Bảng 4-8: Đặc tả UC-09: Giám sát Bot đang chạy (Monitor Live Bot)

4.2.9 UC-10: Xem thị trường (View Market)

Mã Use Case	UC-10
Tên Use Case	View Market
Mô tả Use Case	Cho phép người dùng theo dõi biến động giá của các cặp tiền mã hóa theo thời gian thực thông qua danh sách Market Watch và Biểu đồ nến chuyên sâu. Đồng thời, hệ thống trực quan hóa các điểm vào/ra lệnh của Bot trực tiếp lên biểu đồ để người dùng dễ dàng đánh giá hiệu quả chiến lược.
Tác nhân	User
Mức độ ưu tiên	High
Điều kiện kích hoạt	Người dùng nhấp chọn menu "Giao dịch" trên Sidebar của ứng dụng.
Tiền điều kiện	Người dùng đã đăng nhập thành công. Hệ thống Backend đã kết nối thành công với API của Binance để có thể nhận luồng dữ liệu thị trường.
Hậu điều kiện	Giao diện Trading hiển thị danh sách các cặp coin và biểu đồ nến của cặp coin được chọn, giá cả nhấp nháy cập nhật liên tục (Real-time).
Luồng chính	1. Truy cập màn hình: Người dùng chọn mục "Giao dịch" từ Sidebar. 2. Hiển thị Market Watch: Hệ thống hiển thị Bảng điều khiển thị trường (Market Watch) chứa danh sách các cặp tiền hỗ trợ giao dịch. 3. Chọn Symbol: Người dùng click chọn một cặp tiền (Symbol) cụ thể trong danh sách. 4. Yêu cầu dữ liệu: Frontend yêu cầu Backend cung cấp mảng dữ liệu nến lịch sử và trạng thái Bot tương ứng của Symbol đó. 5. Tổng hợp Markers: Backend lấy dữ liệu nến từ Database, đồng thời trích xuất danh sách các điểm đã khớp lệnh (Trade Markers) của các Bot đang chạy trên cặp tiền này. 6. Vẽ biểu đồ: Hệ thống render Biểu đồ nến và vẽ trực tiếp các điểm khớp lệnh (Mua/Bán) lên Chart để trực quan hóa hành động. 7. Cập nhật Real-time: Dữ liệu giá biến động liên tục được Backend nhận từ sàn Binance và đẩy về Frontend thông qua kết nối WebSocket. Cây nến cuối cùng trên biểu đồ và giá trên bảng Market Watch được cập nhật theo nhịp đập của thị trường.
Luồng thay thế	A1. Thay đổi khung thời gian (Timeframe): Người dùng chọn các khung thời gian khác nhau (1m, 15m, 1h, 1D...) trên thanh công cụ của biểu đồ. Hệ thống gọi lại API để tải mảng dữ liệu nến lịch sử mới tương ứng và vẽ lại Chart. A2. Tìm kiếm cặp tiền: Người dùng nhập tên coin (VD: BTC) vào thanh tìm kiếm trên bảng Market Watch. Hệ thống lọc nhanh và chỉ hiển thị các cặp tiền khớp với từ khóa.
Luồng ngoại lệ	E1. Lỗi tải dữ liệu lịch sử: Backend không phản hồi hoặc Database không có dữ liệu cho Symbol được chọn. Hệ thống hiển thị thông báo "Không thể tải dữ liệu biểu đồ" tại vùng chính giữa màn hình Chart và yêu cầu thử lại sau.

	E2. Mất kết nối luồng giá (WebSocket): Không nhận được giá real-time từ Backend. Hệ thống hiển thị cảnh báo nhỏ (Toast/Icon) "Đang mất kết nối dữ liệu giá..." và tiến hành ngầm thử kết nối lại.
Quy tắc nghiệp vụ	1. Cơ chế luân chuyển dữ liệu: Frontend tuyệt đối không kết nối trực tiếp đến WebSocket của Binance. Dữ liệu giá biến động bắt buộc phải được Backend nhận từ sàn trước, xử lý, sau đó mới đẩy về Frontend qua kênh WebSocket nội bộ. 2. Trực quan hóa chính xác: Các điểm đánh dấu (Markers) thể hiện lệnh phải được gắn chính xác vào tọa độ thời gian (Timestamp) và mức giá (Price) thực tế mà lệnh đó đã khớp trên sàn.
Yêu cầu phi chức năng	1. Tối ưu hiển thị Chart: Bắt buộc sử dụng thư viện Lightweight Charts (do TradingView cung cấp) để render biểu đồ. Thư viện này đảm bảo thao tác thu/phóng (zoom) và kéo/thả (pan) mượt mà, không bị giật lag ngay cả khi Frontend phải render mảng dữ liệu lên đến hàng ngàn cây nến cùng lúc.

Bảng 4-9: Đặc tả UC-10: Xem thị trường (View Market)

4.2.10 UC-11: Xem lịch sử giao dịch (View Trade History)

Mã Use Case	UC-11
Tên Use Case	View Trade History
Mô tả Use Case	Cho phép người dùng tra cứu, phân tích và xuất dữ liệu toàn bộ lịch sử các lệnh giao dịch đã hoàn thành (khớp lệnh thành công, bị thanh lý, hoặc bị hủy) từ tất cả các Bot trong hệ thống. Dữ liệu bao gồm các thông số chi tiết như: Giá khớp, Phí giao dịch (Fee), và Lợi nhuận thực tế (Realized PnL).
Tác nhân	User
Mức độ ưu tiên	Medium
Điều kiện kích hoạt	Người dùng đang ở màn hình "Giao dịch", chọn Tab "Lịch sử giao dịch" nằm phía dưới biểu đồ giá.
Tiền điều kiện	Người dùng đã đăng nhập thành công. Đã có ít nhất một lệnh giao dịch được thực thi và lưu trữ trong Database.
Hậu điều kiện	Danh sách lịch sử lệnh được hiển thị gọn gàng trong không gian Panel, hỗ trợ cuộn và xem lãi/lỗ.
Luồng chính	1. Truy cập khu vực hiển thị: Tại khu vực Panel điều khiển bên dưới Chart, người dùng nhấp chọn Tab "Lịch sử giao dịch". 2. Hiển thị bảng dữ liệu (Data Table): Hệ thống tải và hiển thị danh sách các lệnh đã đóng trong 7 ngày gần nhất dưới dạng bảng thu gọn (Compact Table) phù hợp với chiều cao của Panel. 3. Cấu trúc thông tin: Bảng hiển thị các cột quan trọng nhất: Thời gian, Tên Bot, Cặp tiền (Symbol), Vị thế (Long/Short), Khối lượng, Giá khớp, Phí, Lợi nhuận thực tế (Realized PnL). 4. Sử dụng bộ lọc (Mini-filters): Người dùng sử dụng các bộ lọc dạng Dropdown nhỏ ở góc trên của Tab để lọc nhanh dữ liệu theo: Tên Bot, Symbol, hoặc Trạng thái (Filled/Canceled). 5. Xử lý dữ liệu: Khi thay đổi bộ lọc, Frontend gửi request dạng AJAX xuống Backend, truy vấn Database và cập nhật lại bảng ngay lập tức (không tải lại toàn trang).
Luồng thay thế	A1. Tải thêm dữ liệu (Load More/Infinite Scroll): Khi người dùng cuộn (scroll) danh sách xuống cuối bảng, hệ thống tự động gọi API lấy thêm các trang dữ liệu cũ hơn (Ví dụ: fetch thêm 50 dòng tiếp theo) và nối vào cuối bảng để trải nghiệm xem được liền mạch. A2. Xuất dữ liệu (Export): Người dùng nhấn icon "Xuất CSV" ở góc phải của Tab để tải toàn bộ dữ liệu (theo bộ lọc hiện hành) về máy tính phục vụ cho việc tính toán trên Excel.
Luồng ngoại lệ	E1. Không có dữ liệu: Nếu tài khoản chưa từng chạy Bot hoặc bộ lọc không tìm thấy kết quả khớp, khu vực bảng sẽ hiển thị khoảng trống với thông báo "Chưa có lịch sử giao dịch nào." E2. Lỗi tải dữ liệu: Nếu truy vấn Database thất bại, hiển thị thông báo lỗi màu đỏ tại vùng Tab "Không thể tải lịch sử giao dịch. Vui lòng thử lại sau."
Quy tắc nghiệp vụ	1. Dữ liệu tĩnh (Immutable Data): Khác với Tab "Bot" (dữ liệu nhảy Real-time), dữ liệu hiển thị tại Tab "Lịch sử giao dịch" là dữ liệu đã

	chốt (Realized) trích xuất từ Database hệ thống, đảm bảo đối soát chính xác khoản phí và lợi nhuận thực tế so với trên sàn Binance.
Yêu cầu phi chức năng	1. Hiệu năng truy vấn: Backend cần đánh Index cho các cột user_id, bot_id, và symbol trong bảng lịch sử lệnh để đảm bảo việc cuộn Load More lấy dữ liệu diễn ra dưới 0.5 giây.

Bảng 4-10: Đặc tả UC-11: Xem lịch sử giao dịch (View Trade History)